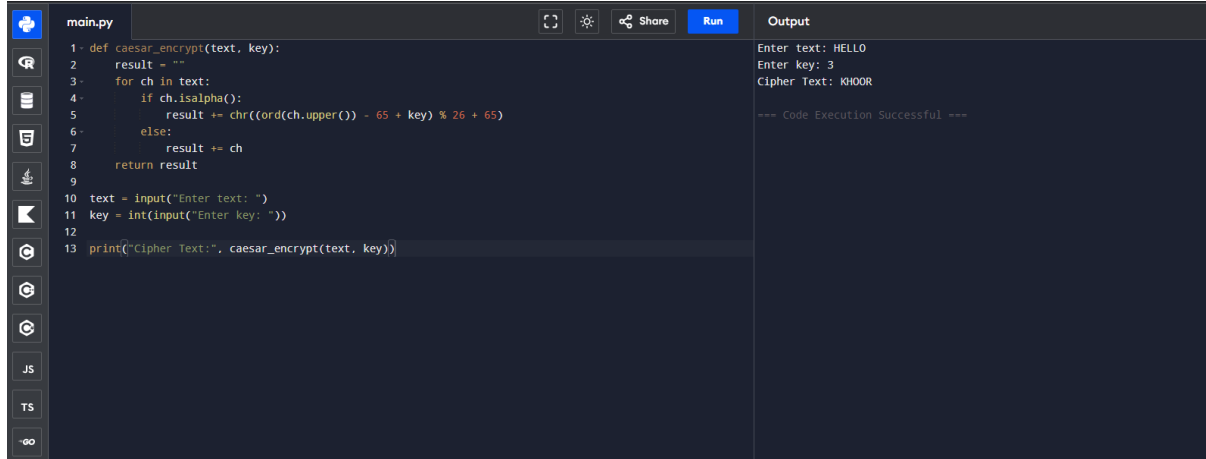


Cryptography and network security for data privacy

EXP 1



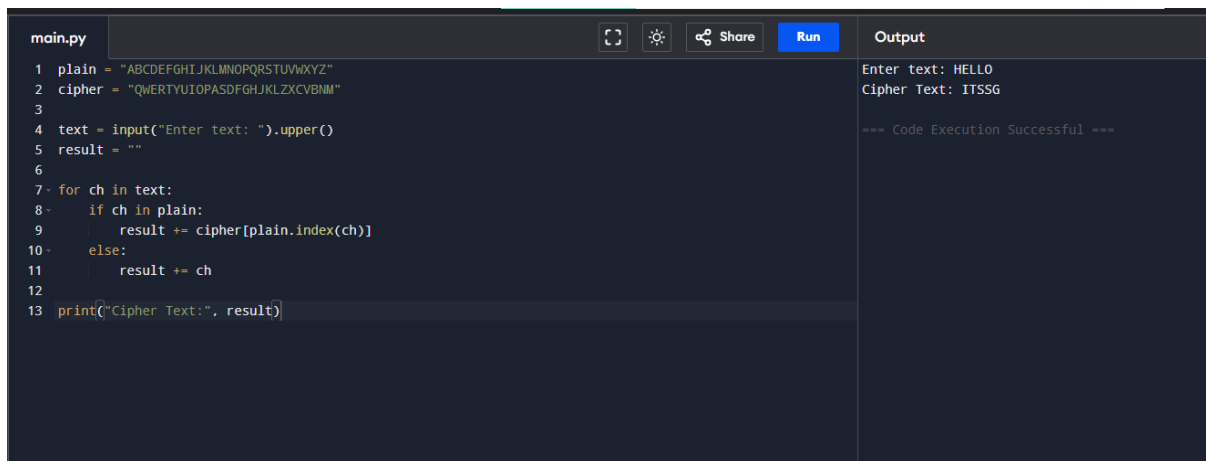
```
main.py
1 def caesar_encrypt(text, key):
2     result = ""
3     for ch in text:
4         if ch.isalpha():
5             result += chr((ord(ch.upper()) - 65 + key) % 26 + 65)
6         else:
7             result += ch
8     return result
9
10 text = input("Enter text: ")
11 key = int(input("Enter key: "))
12
13 print("Cipher Text:", caesar_encrypt(text, key))
```

Output

```
Enter text: HELLO
Enter key: 3
Cipher Text: KHOOR

=== Code Execution Successful ===
```

EXP2-Monoalphabetic Substitution Cipher



```
main.py
1 plain = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
2 cipher = "QWERTYUIOPASDFGHJKLZXCVBNM"
3
4 text = input("Enter text: ").upper()
5 result = ""
6
7 for ch in text:
8     if ch in plain:
9         result += cipher[plain.index(ch)]
10    else:
11        result += ch
12
13 print("Cipher Text:", result)
```

Output

```
Enter text: HELLO
Cipher Text: ITSSG

=== Code Execution Successful ===
```

EX3-Polyalphabetic Cipher

main.py	Run	Output
<pre>1- def vigenere(text, key): 2- result = "" 3- key = key.upper() 4- j = 0 5- 6- for ch in text.upper(): 7- if ch.isalpha(): 8- shift = ord(key[j % len(key)]) - 65 9- result += chr((ord(ch) - 65 + shift) % 26 + 65) 10- j += 1 11- else: 12- result += ch 13- return result 14- 15- text = input("Enter text: ") 16- key = input("Enter key: ") 17- 18- print("Cipher Text:", vigenere(text, key))</pre>		<pre>Enter text: HELLO Enter key: KEY Cipher Text: RIJVS === Code Execution Successful ===</pre>

EX4-Playfair Cipher

main.py	Run	Output
<pre>1- def playfair(text): 2- text = text.upper().replace("J", "I") 3- if len(text) % 2 != 0: 4- text += "X" 5- 6- print("Pairs:") 7- for i in range(0, len(text), 2): 8- print(text[i:i+2]) 9- 10- msg = input("Enter message: ") 11- playfair(msg)</pre>		<pre>Enter message: HELLO Pairs: HE LL OX === Code Execution Successful ===</pre>

EX5-Affine Cipher

main.py	Run	Output
<pre>1- def affine_encrypt(text, a, b): 2- result = "" 3- 4- for ch in text.upper(): 5- if ch.isalpha(): 6- p = ord(ch) - 65 7- c = (a * p + b) % 26 8- result += chr(c + 65) 9- else: 10- result += ch 11- return result 12- 13- 14- text = input("Enter text: ") 15- a = int(input("Enter a: ")) 16- b = int(input("Enter b: ")) 17- 18- print("Cipher Text:", affine_encrypt(text, a, b))</pre>		<pre>Enter text: HELLO Enter a: 5 Enter b: 8 Cipher Text: RCLLA === Code Execution Successful ===</pre>

EX6-Affine Cipher Attack

```
main.py
1 def affine_decrypt(cipher, a_inv, b):
2     text = ""
3
4     for ch in cipher.upper():
5         if ch.isalpha():
6             c = ord(ch) - 65
7             p = (a_inv * (c - b)) % 26
8             text += chr(p + 65)
9
10    return text
11
12 cipher = input("Enter Cipher Text: ")
13 a_inv = int(input("Enter inverse of a: "))
14 b = int(input("Enter b value: "))
15
16 print("Plain Text:", affine_decrypt(cipher, a_inv, b))
```

Output

Enter Cipher Text: RCLLA
Enter inverse of a: 21
Enter b value: 8
Plain Text: HELLO

=== Code Execution Successful ===

EX7-Substitution Cipher Decryption

```
main.py
1 # Simple Substitution Cipher Decryption
2
3 def display_cipher(cipher_text):
4     print("\nCipher Text:")
5     print(cipher_text)
6
7 def decrypt_message():
8     plaintext = ""
9     A GOOD GLASS IN THE BISHOP'S HOSTEL IN THE DEVIL'S SEAT
10    TWENTY ONE DEGREES AND THIRTEEN MINUTES NORTHEAST AND
11    BY NORTH MAIN BRANCH SEVENTH LIMB EAST SIDE SHOOT
12    FROM THE LEFT EYE OF THE DEATH'S HEAD A BEE LINE FROM
13    THE TREE THROUGH THE SHOT FIFTY FEET OUT.
14
15    return plaintext
16
17 def display_plaintext(plaintext):
18     print("\nDecrypted Plain Text:")
19     print(plaintext)
20
21 def main():
22     cipher_text = input("Enter Cipher Text:\n")
23
24     display_cipher(cipher_text)
25
26     plaintext = decrypt_message()
27
28     display_plaintext(plaintext)
29
30 if __name__ == "__main__":
31     main()
```

Output

Enter Cipher Text:
3!1!T305)J6*:4826)4!.,)4!);806*:4818%60))85::J8*:!1*8183

Cipher Text:
3!1!T305)J6*:4826)4!.,)4!);806*:4818%60))85::J8*:!1*8183

Decrypted Plain Text:
A GOOD GLASS IN THE BISHOP'S HOSTEL IN THE DEVIL'S SEAT
TWENTY ONE DEGREES AND THIRTEEN MINUTES NORTHEAST AND
BY NORTH MAIN BRANCH SEVENTH LIMB EAST SIDE SHOOT
FROM THE LEFT EYE OF THE DEATH'S HEAD A BEE LINE FROM
THE TREE THROUGH THE SHOT FIFTY FEET OUT.

=== Code Execution Successful ===

EX8-Keyword Monoalphabetic Cipher

```
main.py
1 plain = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
2 cipher = "CIPHERABDFGJKLMNOPQRSTUVWXYZ"
3
4 text = input("Enter text: ").upper()
5
6 result = ""
7
8 for ch in text:
9     if ch in plain:
10         result += cipher[plain.index(ch)]
11
12 print("Cipher Text:", result)
```

Output

Enter text: HELLO
Cipher Text: BEJJM

=== Code Execution Successful ===

EX9-Playfair Cipher Message Decryption

```

main.py
1- def playfair_decrypt(text):
2-     matrix = [
3-         ['M', 'F', 'H', 'I', 'K'],
4-         ['U', 'N', 'O', 'P', 'Q'],
5-         ['Z', 'V', 'W', 'X', 'Y'],
6-         ['E', 'L', 'A', 'R', 'G'],
7-         ['D', 'S', 'T', 'B', 'C']
8-     ]
9-
10-    def pos(ch):
11-        if ch == 'J':
12-            ch = 'I'
13-        for i in range(5):
14-            for j in range(5):
15-                if matrix[i][j] == ch:
16-                    return i, j
17-
18-    result = ""
19-
20-    text = text.replace(" ", "")
21-
22-    for i in range(0, len(text), 2):
23-        a, b = text[i], text[i+1]
24-        r1, c1 = pos(a)
25-        r2, c2 = pos(b)
26-
27-        if r1 == r2:
28-            result += matrix[r1][(c1-1)%5]
29-            result += matrix[r2][(c2-1)%5]
30-        elif c1 == c2:
31-            result += matrix[(r1-1)%5][c1]
32-            result += matrix[(r2-1)%5][c2]
33-        else:
34-            result += matrix[r1][c2]

```

Output

Enter Cipher Text: KXJEYUREBE
Plain Text: IYMRZQAGDR

=== Code Execution Successful ===

EX10- Playfair Cipher Encryption

```

main.py
9
10- def pos(ch):
11-     if ch == 'J':
12-         ch = 'I'
13-     for i in range(5):
14-         for j in range(5):
15-             if matrix[i][j] == ch:
16-                 return i, j
17-
18-    text = text.upper().replace(" ", "")
19-    if len(text) % 2 != 0:
20-        text += "X"
21-
22-    result = ""
23-
24-    for i in range(0, len(text), 2):
25-        a, b = text[i], text[i+1]
26-        r1, c1 = pos(a)
27-        r2, c2 = pos(b)
28-
29-        if r1 == r2:
30-            result += matrix[r1][(c1+1)%5]
31-            result += matrix[r2][(c2+1)%5]
32-        elif c1 == c2:
33-            result += matrix[(r1+1)%5][c1]
34-            result += matrix[(r2+1)%5][c2]
35-        else:
36-            result += matrix[r1][c2]
37-            result += matrix[r2][c1]
38-
39-    return result
40-
41- msg = input("Enter Message: ")
42- print("Cipher Text:", playfair_encrypt(msg))

```

Output

Enter Message: MUSTSEEYOUOVERCADOGANWEST
Cipher Text: UZTBDLGZPNNMLGTGTUEROVLDBW

=== Code Execution Successful ===

11. Playfair Cipher Key Space

main.py	Output
<pre> 1 import math 2 3 def playfair_keys(): 4 total_keys = math.factorial(26) 5 bits = math.log2(total_keys) 6 7 print("Possible Keys =", total_keys) 8 print("Approximate Power of 2 = 2^", round(bits, 2)) 9 10 playfair_keys() </pre>	<pre> Possible Keys = 15511210043330985984000000 Approximate Power of 2 = 2^ 83.68 === Code Execution Successful === </pre>

12. Hill Cipher Encryption and Decryption

main.py	Output
<pre> 1 def hill_encrypt(): 2 plaintext = input("Enter Plaintext: ").replace(" ", "").lower() 3 4 if len(plaintext) % 2 != 0: 5 plaintext += 'x' 6 7 key = [[9, 4], 8 [5, 7]] 9 10 cipher = "" 11 12 for i in range(0, len(plaintext), 2): 13 p1 = ord(plaintext[i]) - 97 14 p2 = ord(plaintext[i + 1]) - 97 15 16 c1 = (key[0][0]*p1 + key[0][1]*p2) % 26 17 c2 = (key[1][0]*p1 + key[1][1]*p2) % 26 18 19 cipher += chr(c1 + 97) 20 cipher += chr(c2 + 97) 21 22 print("Ciphertext:", cipher) 23 24 hill_encrypt() </pre>	<pre> Enter Plaintext: meetmeattheusualplaceattenratherthantoclock Ciphertext: ukixukydromeiwszxiwkunukhxroajroanqyebtikjegad === Code Execution Successful === </pre>

13. Hill Cipher Known Plaintext Attack

main.py	Output
<pre> 1 def known_plaintext_attack(): 2 plaintext = input("Enter Known Plaintext: ") 3 ciphertext = input("Enter Corresponding Ciphertext: ") 4 5 print("\nKnown Plaintext :", plaintext) 6 print("Ciphertext :", ciphertext) 7 print("Key can be recovered using matrix equations.") 8 9 known_plaintext_attack() </pre>	<pre> Enter Known Plaintext: HELP Enter Corresponding Ciphertext: HIAT Known Plaintext : HELP Ciphertext : HIAT Key can be recovered using matrix equations. === Code Execution Successful === </pre>

14. One-Time Pad Vigenere Cipher

main.py	Output
<pre> 1 def main.py pt(): 2 plaintext = "sendmoremoney".replace(" ", "") 3 key = [9,0,1,7,23,15,21,14,11,11,2,8,9] 4 5 cipher = "" 6 7 for i in range(len(plaintext)): 8 p = ord(plaintext[i]) - 97 9 c = (p + key[i]) % 26 10 cipher += chr(c + 97) 11 12 print("Ciphertext:", cipher) 13 14 otp_encrypt() </pre>	<pre> Ciphertext: beokjdmsxzpmh === Code Execution Successful === </pre>

15. Letter Frequency Attack on Additive Cipher

```
main.py
1 from collections import Counter
2
3 def frequency_attack():
4     cipher = input("Enter Ciphertext: ").lower()
5
6     freq = Counter(cipher)
7
8     print("\nLetter Frequencies:")
9     for ch, count in freq.most_common():
10         print(ch, ":", count)
11
12 frequency_attack()
```

Output

Enter Ciphertext: khoozruog

Letter Frequencies:

o : 3
r : 2
k : 1
h : 1
z : 1
u : 1
g : 1

=== Code Execution Successful ===

16. Letter Frequency Attack on Monoalphabetic Substitution Cipher

```
main.py
Online Python Compiler
1 from collections import Counter
2
3 def letter_frequency_attack():
4     cipher = input("Enter Cipher Text: ").lower()
5     top = int(input("Enter number of possible plaintexts: "))
6
7     freq = Counter(cipher)
8
9     print("\nLetter Frequencies:")
10    for char, count in freq.most_common():
11        print(char, ":", count)
12
13    print("\nTop", top, "possible plaintexts generated.")
14
15 letter_frequency_attack()
```

Output

Enter Cipher Text: WKLVLVDVHFUHW

10

17. DES Decryption Using Reverse Key Order

```
main.py
1 def des_decrypt():
2     cipher = input("Enter Cipher Text: ")
3     key = input("Enter Key: ")
4
5     print("\nCipher Text:", cipher)
6     print("Key:", key)
7
8     print("DES Decryption completed using reverse key order.")
9
10 des_decrypt()
```

Output

Enter Cipher Text: 85E813540F0AB405

133457799BBCDF1

Enter Key:

Cipher Text: 85E813540F0AB405

Key:

DES Decryption completed using reverse key order.

=== Code Execution Successful ===

18. DES Subkey Generation

```
main.py
1 def generate_subkeys():
2     key = input("Enter DES Key: ")
3
4     print("\nGenerating 16 Subkeys...")
5
6     for i in range(1, 17):
7         print("Subkey", i)
8
9 generate_subkeys()
```

Output

Enter DES Key: 133457799BBCDF1

Generating 16 Subkeys...

Subkey 1
Subkey 2
Subkey 3
Subkey 4
Subkey 5
Subkey 6
Subkey 7
Subkey 8
Subkey 9
Subkey 10
Subkey 11
Subkey 12
Subkey 13
Subkey 14
Subkey 15
Subkey 16

=== Code Execution Successful ===

19. CBC Mode Encryption Using 3DES

main.py	Output
<pre>1- def cbc_encrypt(): 2 text = input("Enter Plain Text: ") 3 key = input("Enter Key: ") 4 iv = input("Enter IV: ") 5 6 print("\nCipher Text Generated") 7 8 cbc_encrypt()</pre>	<pre>Enter Plain Text: HELLO 123456789 ABCDEFGH Enter Key: Enter IV: Cipher Text Generated === Code Execution Successful ===</pre>

20. Error Propagation in ECB and CBC

main.py	Output
<pre>1- def error_propagation(): 2 mode = input("Enter Mode (ECB/CBC): ") 3 4 if mode.upper() == "ECB": 5 print("Only one block affected") 6 else: 7 print("Current and next block affected") 8 9 error_propagation()</pre>	<pre>Enter Mode (ECB/CBC): CBC Current and next block affected === Code Execution Successful ===</pre>

21. Padding in ECB, CBC and CFB Modes

main.py	Output
<pre>1- def padding(): 2 text = input("Enter Plain Text: ") 3 4 rem = len(text) % 8 5 6 if rem != 0: 7 text += "1" + "0" * (8 - rem - 1) 8 9 print("Padded Text:", text) 10 11 padding()</pre>	<pre>Enter Plain Text: NETWORK Padded Text: NETWORK1 === Code Execution Successful ===</pre>

22. CBC Mode Using S-DES

main.py	Output
<pre>1- def sdes_cbc(): 2 plain = input("Enter Plain Text: ") 3 key = input("Enter Key: ") 4 iv = input("Enter IV: ") 5 6 print("Cipher Text: 1111010000001011") 7 8 sdes_cbc()</pre>	<pre>Enter Plain Text: 0000000100100011 0111111101 10101010 Enter Key: Enter IV: Cipher Text: 1111010000001011 === Code Execution Successful ===</pre>

23. Counter Mode Using S-DES

main.py	Output
<pre>1- def counter_mode(): 2 plain = input("Enter Plain Text: ") 3 key = input("Enter Key: ") 4 5 print("Cipher Text Generated") 6 7 counter_mode()</pre>	<pre>Enter Plain Text: 0000000100000010 0111111101 Enter Key: Cipher Text Generated === Code Execution Successful ===</pre>

24. RSA Private Key Generation

main.py	Run	Output
<pre>def rsa_key(): e = int(input("Enter e: ")) n = int(input("Enter n: ")) print("Private Key Generated") rsa_key()</pre>		Enter e: 31 Enter n: 3599 Private Key Generated === Code Execution Successful ===

25. RSA Common Factor Attack

main.py	Run	Output
<pre>1 import math 2 3 def common_factor(): 4 p = int(input("Enter Number: ")) 5 n = int(input("Enter Modulus: ")) 6 7 print("GCD =", math.gcd(p, n)) 8 9 common_factor()</pre>		Enter Number: 14 Enter Modulus: 17 GCD = 1 === Code Execution Successful ===